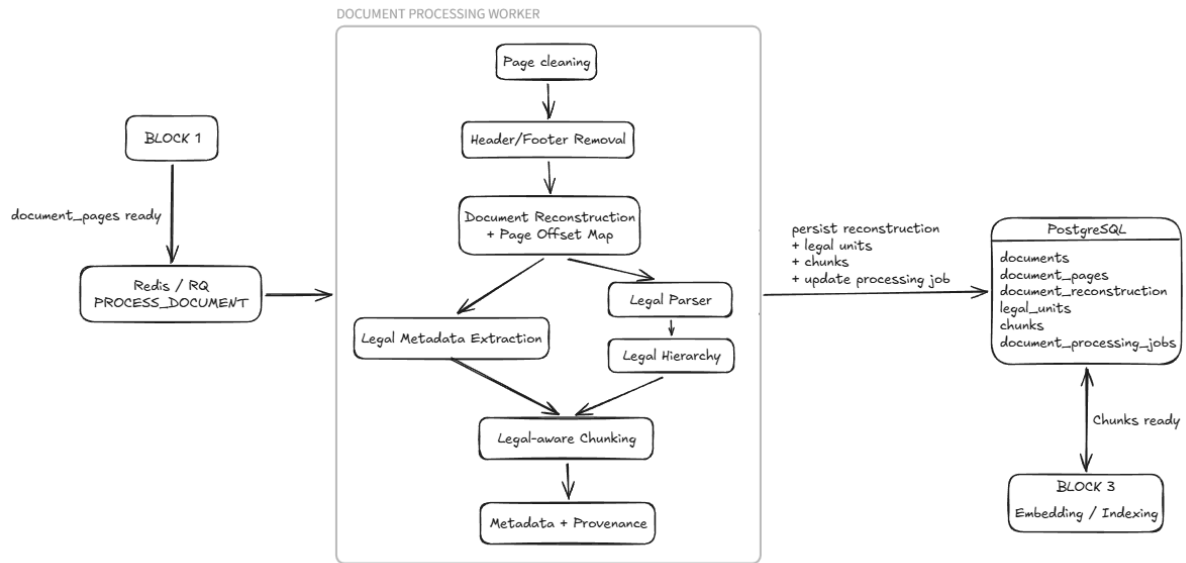


BLOCK 2



Block 1 hoàn thành và có **document_pages**.

Hệ thống tạo một job **PROCESS_DOCUMENT** vào **Redis / RQ**.

Document Processing Worker consume job đó và bắt đầu xử lý tài liệu.

Worker thực hiện lần lượt:

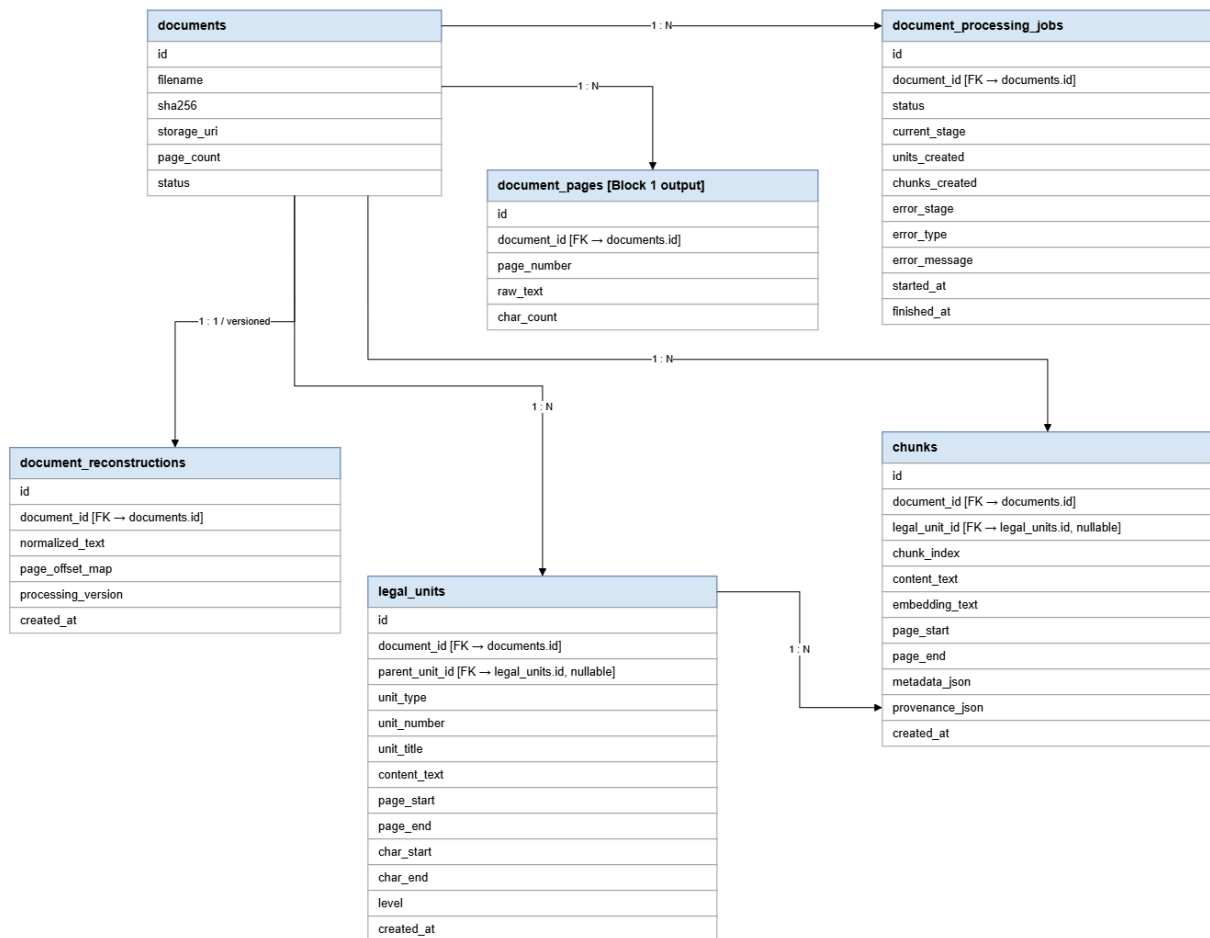
- Page Cleaning
- Header / Footer Removal
- Document Reconstruction + Page Offset Map
- Legal Metadata Extraction
- Legal Parser → Legal Hierarchy
- Legal-aware Chunking
- Metadata + Provenance

Sau đó Worker lưu kết quả vào **PostgreSQL**:

- **document_reconstructions**
- **legal_units**
- **chunks**
- **document_processing_jobs**

Khi **chunks** đã sẵn sàng thì Block 3 có thể dùng chúng để Embedding / Indexing.

BLOCK 2 — POSTGRESQL / DATA CONTRACT



documents Record gốc của tài liệu. Đây là entity cha.

document_pages Output của Block 1. Giữ text thô theo từng trang:

document_processing_jobs Theo dõi riêng quá trình xử lý của Block 2.

Ví dụ:

status = PROCESSING

current_stage = LEGAL_PARSING

Nếu lỗi Block 2, đây là bảng đầu tiên cần xem.

document_reconstructions Giữ output sau khi clean + merge pages.

Quan trọng nhất:

- `normalized_text`
- `page_offset_map`

Nó cho phép debug: text bị sai từ Reconstruction hay Parser sai?

legal_units Giữ cấu trúc pháp lý đã parse ra.

Ví dụ: Chapter → Article → Clause → Point

Đây chính là cây pháp lý của tài liệu.

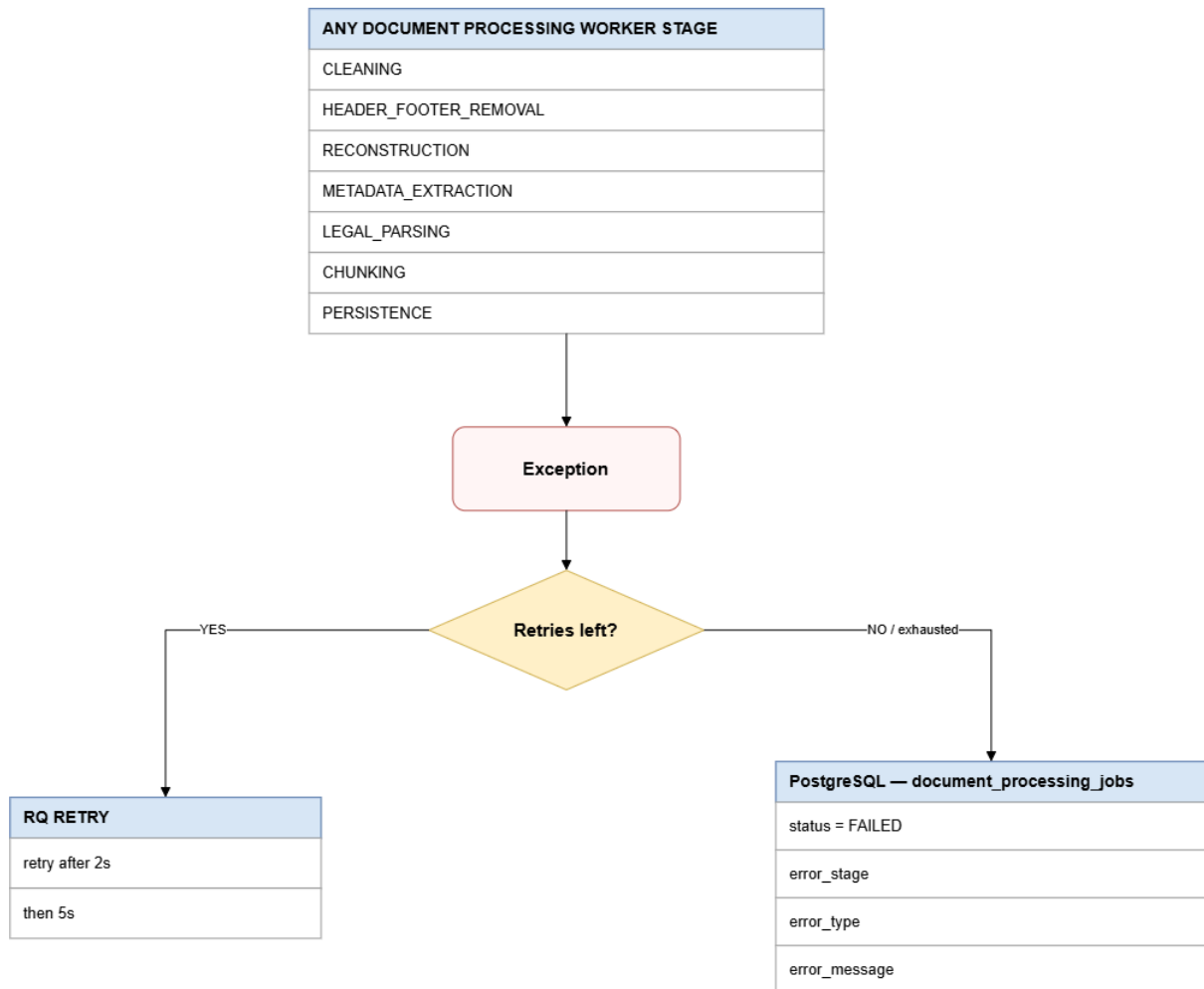
chunks

Output cuối Block 2.

Giữ:

- `content_text`
- `embedding_text`
- `page_start/page_end`
- `metadata`
- `provenance`
- link về `legal_unit_id`

BLOCK 2 — FAILURE / RETRY LANE



Hình này mô tả: **Block 2 xử lý lỗi như thế nào?**

Lỗi có thể xuất hiện ở:

CLEANING
HEADER_FOOTER_REMOVAL
RECONSTRUCTION
METADATA_EXTRACTION
LEGAL_PARSING
CHUNKING
PERSISTENCE

Nếu có exception: Exception -> Retries left?

Nếu còn retry YES -> RQ retry -> đợi 2s -> retry -> nếu cần đợi thêm 5s

Không đánh dấu **FAILED** ngay vì có thể chỉ là lỗi tạm thời.

Nếu hết retry NO / exhausted -> PostgreSQL

lưu:

status = FAILED

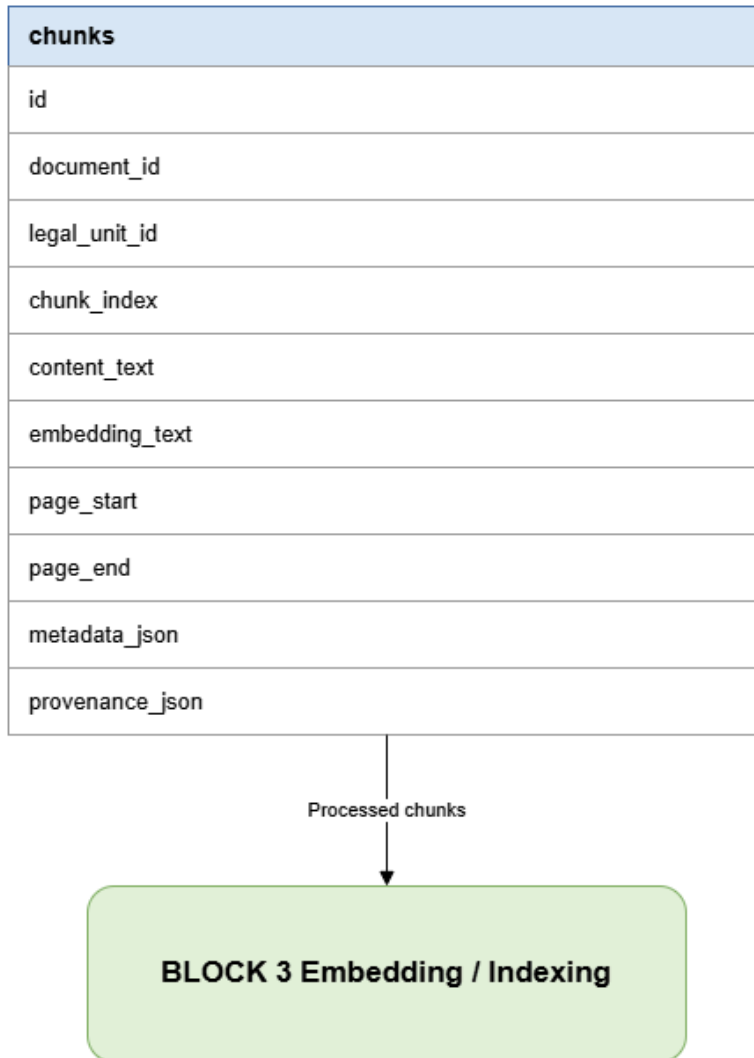
error_stage

error_type

error_message

Nhờ vậy khi debug có thể biết chính xác: *Block 2 fail ở Reconstruction, Legal Parsing hay Persistence?*

BLOCK 2 — OUTPUT CONTRACT

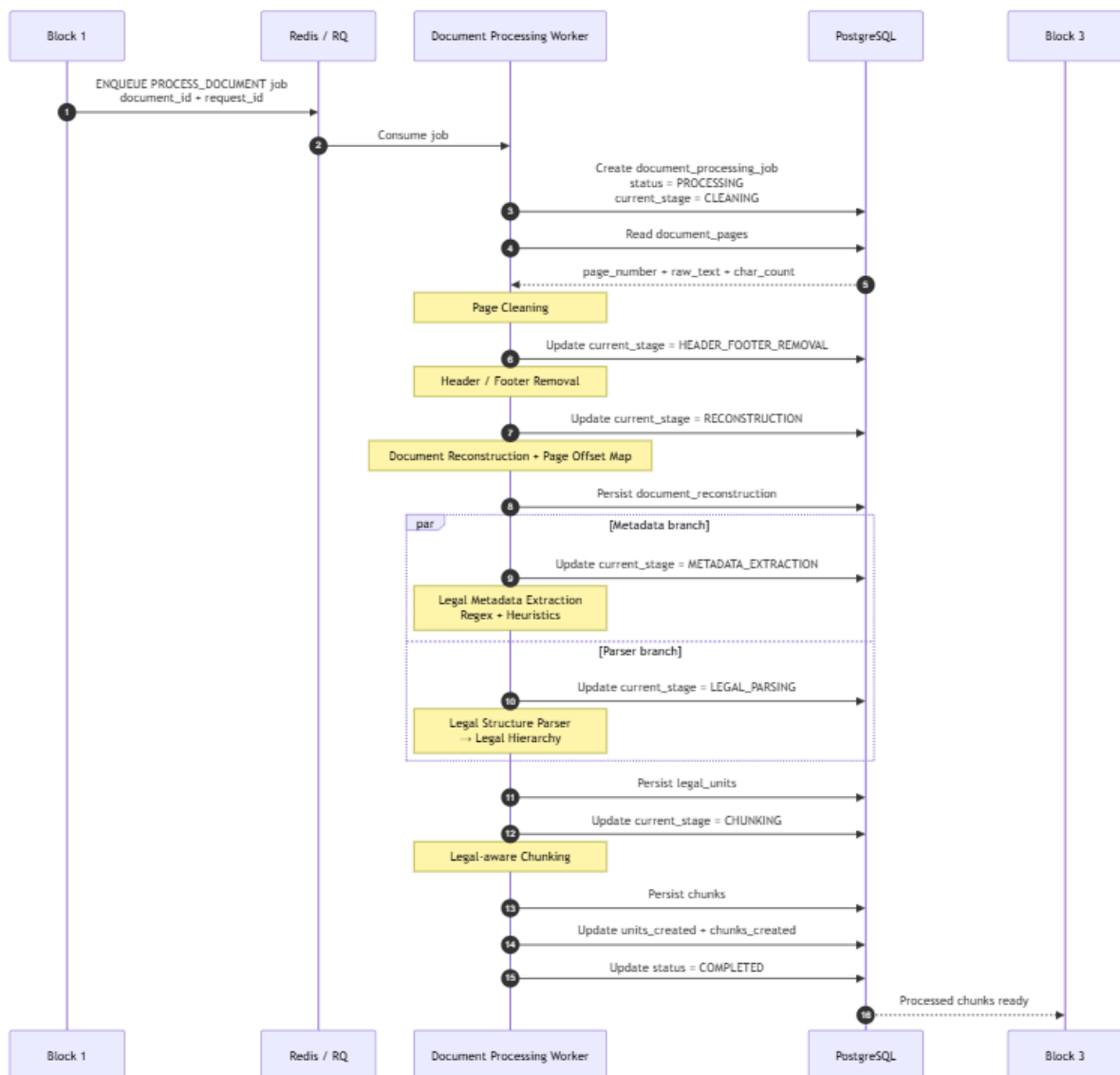


Hình này trả lời: ***Block 2 cam kết giao chính xác dữ liệu gì cho Block 3?***

Output là: chunks

Mỗi chunk có:

- **document_id** Chunk thuộc tài liệu nào.
- **legal_unit_id** Chunk thuộc Điều/Khoản/Điểm nào.
- **chunk_index** Thứ tự chunk trong tài liệu.
- **content_text** Text thật dùng để hiển thị / trả source.



- **Block 1 → Redis/RQ:** sau khi `document_pages` đã sẵn sàng, hệ thống enqueue một `PROCESS_DOCUMENT` job chứa `document_id`, `request_id` và thông tin job cần thiết.
- **Redis/RQ → Document Processing Worker:** Worker lấy job khỏi queue và bắt đầu xử lý tài liệu.
- **Worker → PostgreSQL:** tạo `document_processing_job`, đổi trạng thái thành `PROCESSING` và đặt `current_stage = CLEANING`.
- **Worker → PostgreSQL:** đọc toàn bộ `document_pages` của tài liệu, gồm `page_number`, `raw_text`, `char_count`.

- **Worker xử lý nội bộ — Page Cleaning:** làm sạch text cơ bản như chuẩn hóa Unicode, whitespace và các lỗi format nhỏ mà không làm mất cấu trúc pháp lý.
- **Worker → PostgreSQL:** cập nhật `current_stage = HEADER_FOOTER_REMOVAL`.
- **Worker xử lý nội bộ — Header/Footer Removal:** loại các header, footer, page number lặp lại giữa các trang.
- **Worker → PostgreSQL:** cập nhật `current_stage = RECONSTRUCTION`.
- **Worker xử lý nội bộ — Document Reconstruction:** ghép các trang thành một continuous document, xử lý nội dung bị vắt qua trang và đồng thời tạo `page_offset_map` để vẫn truy ngược được text thuộc trang nào.
- **Worker → PostgreSQL:** persist kết quả reconstruction vào `document_reconstructions`.
- **Worker xử lý nội bộ — Legal Metadata Extraction:** dùng Regex + Heuristics để lấy metadata toàn văn bản như loại văn bản, số hiệu, tiêu đề/trích yếu, cơ quan ban hành, ngày ban hành...
- **Worker xử lý nội bộ — Legal Structure Parser:** nhận diện các cấu trúc pháp lý như `Chương`, `Mục`, `Điều`, `Khoản`, `Điểm`.
- **Worker xử lý nội bộ — Legal Hierarchy:** từ các thành phần parser nhận diện được, xây quan hệ cha-con, ví dụ `Chương → Điều → Khoản → Điểm`.
- **Worker → PostgreSQL:** persist các cấu trúc đã parse vào `legal_units`.
- **Worker → PostgreSQL:** cập nhật `current_stage = CHUNKING`.
- **Worker xử lý nội bộ — Legal-aware Chunking:** chia text thành chunk dựa ưu tiên vào boundary pháp lý như Điều/Khoản/Điểm, chỉ dùng sentence/token split khi một legal unit quá dài.
- **Worker xử lý nội bộ — Metadata + Provenance:** gán cho từng chunk thông tin như `document_id`, `legal_unit_id`, `page_start`, `page_end`, metadata pháp lý và thông tin truy vết nguồn.
- **Worker → PostgreSQL:** persist các chunk vào bảng `chunks`, đồng thời cập nhật `units_created` và `chunks_created`.

- **Worker → PostgreSQL:** khi toàn bộ processing thành công, đổi `document_processing_job.status = COMPLETED`.
- **PostgreSQL → Block 3:** các `processed chunks` đã sẵn sàng để Block 3 thực hiện Embedding / Indexing.